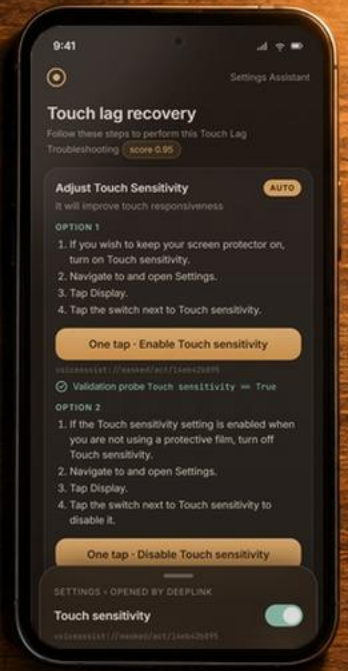
TapTrace

SAMSUNG PRISM · GENAI HACKATHON 3.0

From a vague complaint to a verified fix, in one tap.

A grounded troubleshooting engine for Samsung PRISM Theme 02: every step traced to its source, every Settings fix one exact deeplink away, every repeat answered from cache in milliseconds.



- Evidence-locked steps
- Exact-screen deeplinks
- Relevance gate
- Contrastive cache

THEME ID 02 | TEAM TAPTRACE | THAPAR INSTITUTE OF ENGINEERING & TECHNOLOGY | ARNAV JOSHI · ajoshi4_be23@thapar.edu

CODE github.com/Arnav020/Tap-Trace | LIVE DEMO taptrace.onrender.com

Vague complaints meet adversarial data: six caveats shaped every design decision.

Found by auditing all 578 catalog rows and 20 knowledge articles before writing any code.

THE PROBLEM

"My Nexa X1 screen inputs are delayed and the touch responsiveness is laggy..."

Today an agent reads the article, picks and orders the steps by hand (about 15 minutes per case), and the customer still hunts through nested Settings menus.

WHAT THE BRIEF DEMANDS

- Ordered steps, only from the article
- The exact Settings screen, one tap
- Repeats in ≤ 300 ms, new in ≤ 8 s
- Strict JSON contract, zero URL leaks

21
TV and appliance screens hidden in the phone catalog
→ Catalog Compiler quarantines them

14
catalog labels that name the wrong feature
→ Match screens, never trust labels

0.94
similarity of "won't turn ON" and "won't turn OFF"
→ Polarity-aware resolver and cache

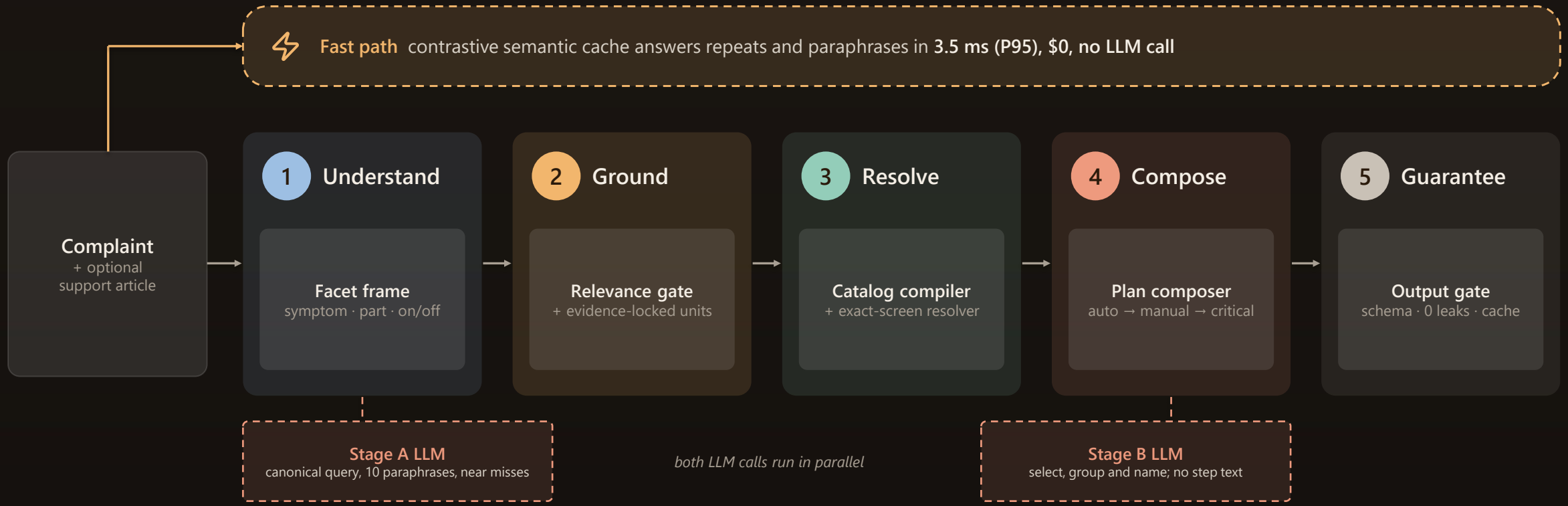
4 / 20
knowledge articles are the wrong document
→ Relevance gate returns no_match

DL-0022
is nearest to "factory reset", but a different screen
→ Abstain instead of guessing

12 words
in the official sample's description (5–7 allowed)
→ Every field rule enforced in code

Five stages: the LLM may select and name steps, but it never writes one.

Everything except the two LLM calls is deterministic, tested code (27 automated tests).



3.8 s new complaint, P95 (target 8 s)	**3.5 ms** repeat, P95 (target 300 ms)	**$0.0003** per new complaint; repeats $0	**100% · 0** schema-valid · URL leaks

Four modules carry the weight, and each one is proven by a measurement.

Measured by scripts/evaluate.py on the 20 provided scenarios and a frozen, never-tuned test set.

Evidence-locked extraction

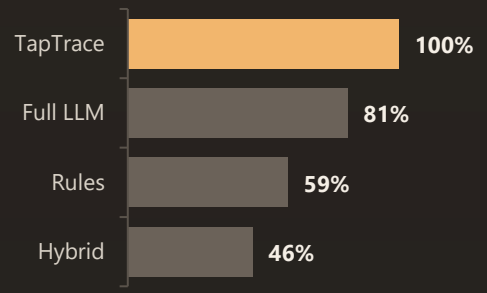
Steps are cut from the article's own sentences and keep their source id. The LLM only selects and names them.

3.00 / 3
step accuracy vs hand-annotated gold

0
URL leaks across all outputs

Catalog compiler + resolver

578 noisy rows become 397 screens × on/off/open/set. Exact leaf label, then fuzzy with a margin, else abstain.



exact deeplinks, 37-case benchmark

Relevance gate

Compares the complaint's symptom and context with the article before extracting anything.

4 / 20
wrong articles answered with no_match

100%
fit decisions match gold

Contrastive semantic cache

Each entry learns its own threshold from paraphrases and near misses; a symbolic guard blocks on/off flips.

80%
unseen paraphrases hit (frozen test)

32% → 4.5%
false hits vs one global threshold (dev)

The same engine fits four Galaxy support touchpoints, from call centre to device.

Mapped from the brief's own inputs: SIIS articles, bixby:// Settings deeplinks and validation probes.

Customer-care agents
SIIS knowledge base

The agent pastes the complaint and gets an ordered plan, each step highlighted in the internal article.

`complaint + article → plan + evidence`

~15 min of manual triage → 1.3 s (P50)

Samsung Members
self-service help

The user describes the problem in the app and gets one-tap fixes; the validation probe confirms each one.

`complaint → one tap → probe ✓`

No more hunting through Settings

Bixby voice
spoken complaints

"My touch is laggy" becomes a plan, and Bixby opens the exact screen through the same deeplink scheme.

`speech → text → plan → deeplink`

Voice in, verified tap out

On-device worklet
Galaxy offline support

The 23 MB INT8 embedder, compiled catalog and cache run on CPU in about 150 MB of RAM.

`cache on device; LLM only for new`

Known issues answered offline

Why it transfers: the deeplink scheme, brand names and article format are configuration, not code (the provided data is masked as TechCorp / Nexa and voiceassist://). Next step: use validation-probe results as feedback on which fixes really work.

Covering a new scenario takes a knowledge article, not new code.

Never-seen Battery, Camera, Performance and Wi-Fi articles ran through the same pipeline unchanged.

HOW A NEW SCENARIO GETS COVERED

1. **A new complaint and its article arrive**
   gate and step extraction run automatically
2. **Settings steps map to exact screens**
   compiled catalog; no retraining, no new rules
3. **One cold request, 1.3 s (P50)**
   about $0.0003 of LLM cost
4. **The plan is cached**
   every repeat and paraphrase: 3.5 ms (P95), $0

UNSEEN-DOMAIN TEST

8 / 8

Settings steps mapped to the exact screen with the right on/off variant, in articles the engine had never seen

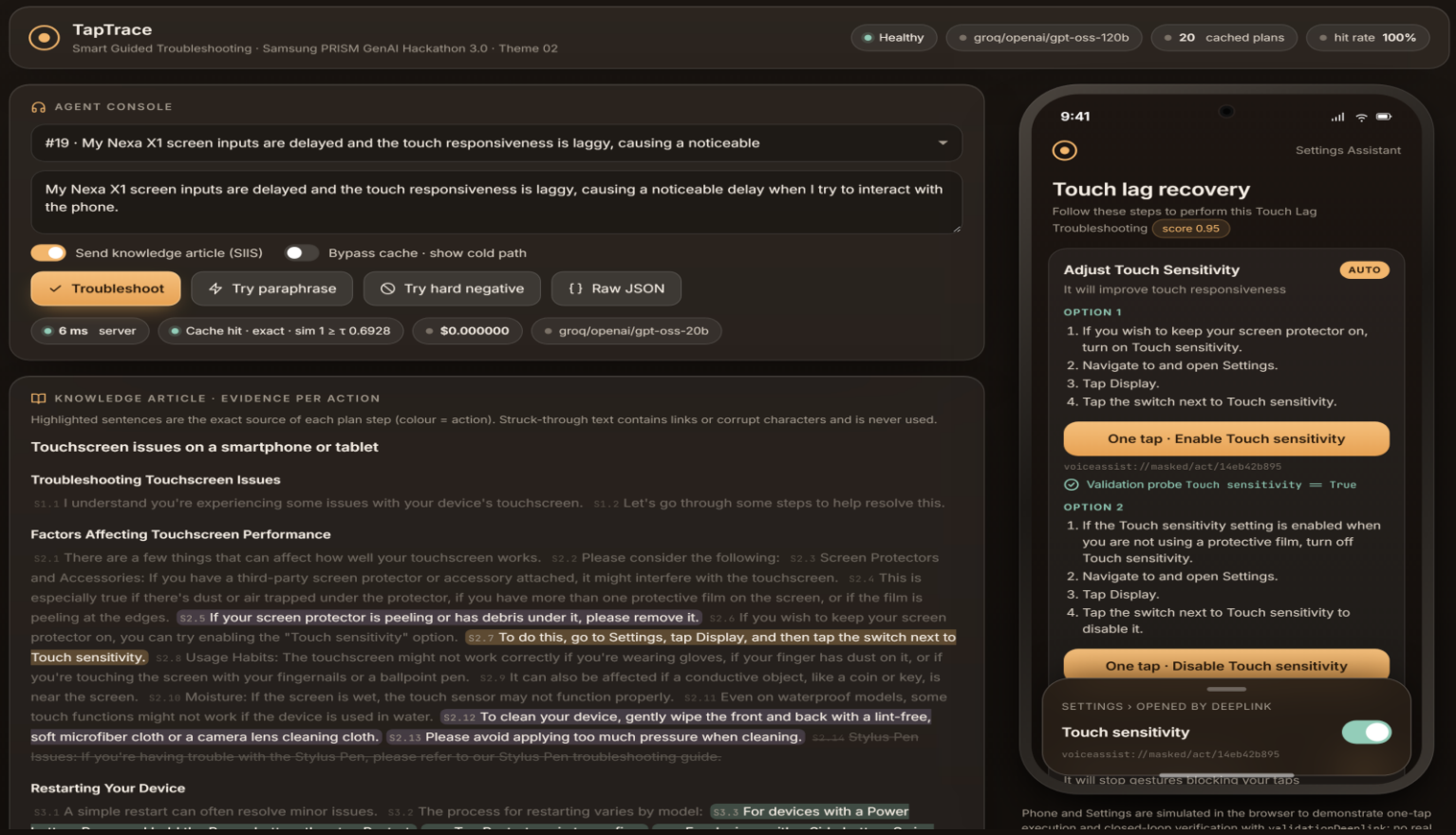
Battery | Camera | Performance | Wi-Fi

EXTENSION POINTS

New article or scenario	**automatic**
New Settings screens	**re-run the Catalog Compiler**
New vocabulary	**facet lexicon entries (data)**
10,000 scenarios	**≈ $3 one-time LLM cost**
Serving	**495 req/s on one CPU container**

Live on the deployed site: one complaint, from article to verified tap.

Screen recording of taptrace.onrender.com, 1:51.



WHAT TO WATCH FOR

0:00 — **A new complaint, end to end**
evidence highlighted in the article, cost shown

0:36 — **One tap + validation probe**
exact Settings screen opens; the probe confirms

1:00 — **Paraphrase hit, near miss refused**
milliseconds from cache; the guard says why

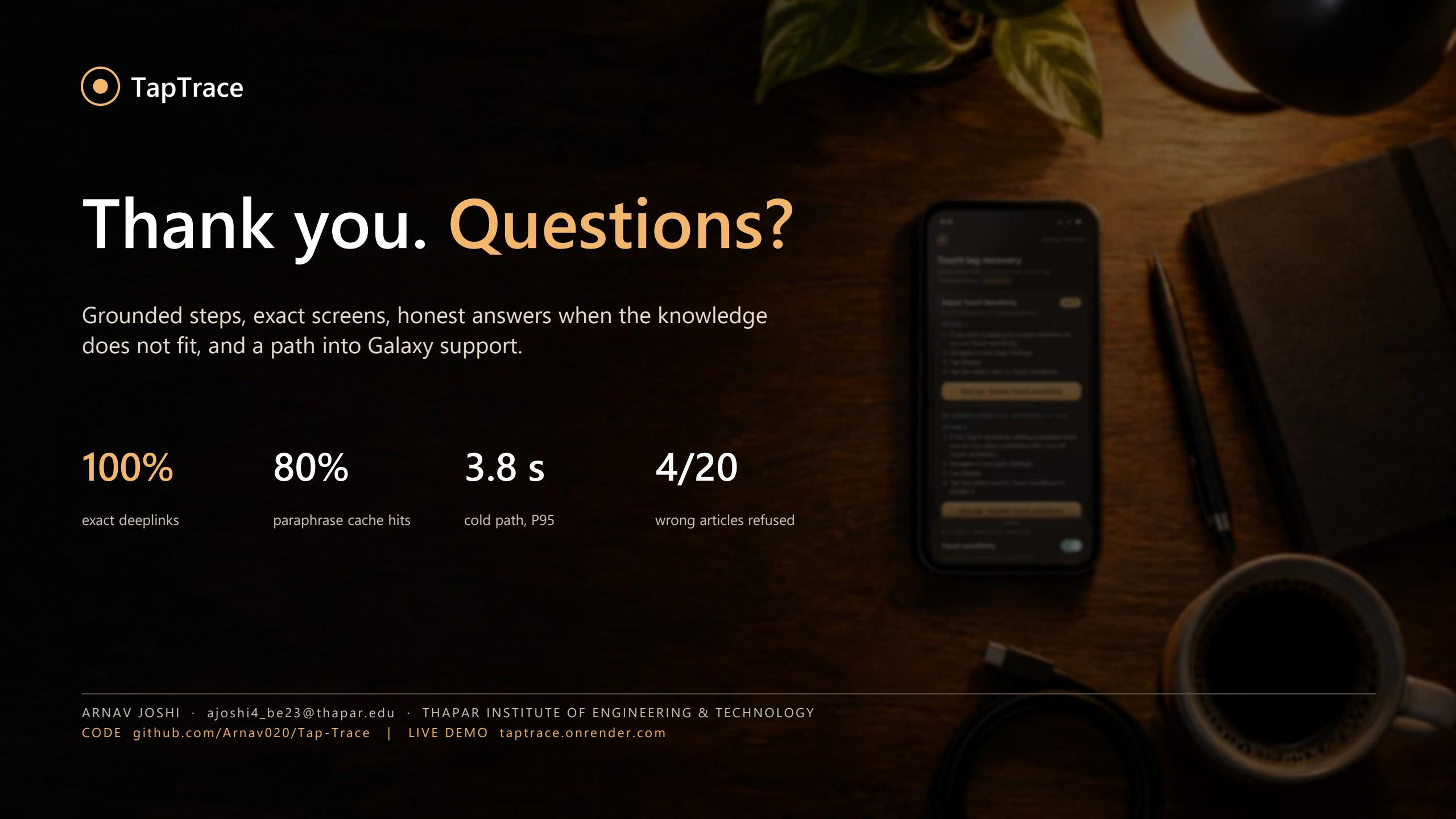
1:32 — **Wrong article → no_match**
an empty plan instead of invented steps

TapTrace

Thank you. Questions?

Grounded steps, exact screens, honest answers when the knowledge does not fit, and a path into Galaxy support.

100%	80%	3.8 s	4/20
exact deeplinks	paraphrase cache hits	cold path, P95	wrong articles refused



ARNAV JOSHI · ajoshi4_be23@thapar.edu · THAPAR INSTITUTE OF ENGINEERING & TECHNOLOGY

CODE github.com/Arnav020/Tap-Trace | LIVE DEMO taptrace.onrender.com